

Clase 108 — Vulnerabilidades en carga de archivos

Parte: 4 — Seguridad de aplicaciones web · Fuente: *The Web Application Hacker's Handbook / Bug Bounty Bootcamp (Vickie Li)* 🕒 Duración estimada: 100 min · Nivel: Avanzado

🎯 Objetivo

Explotar fallos en la **carga de archivos (file upload)**: subir contenido malicioso que la aplicación acepta y ejecuta o sirve de forma peligrosa. Un upload mal validado puede convertirse en web shell (RCE), XSS almacenado, path traversal o SSRF.

⚠️ **Ética:** subir una web shell equivale a RCE. Solo en labs propios/autorizados (DVWA, PortSwigger). Nunca en sistemas ajenos.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Identificar** qué validaciones aplica un formulario de subida.
2. **Evadir** filtros por extensión, Content-Type y magic bytes.
3. **Conseguir** ejecución subiendo una web shell en un lab.
4. **Explotar** upload para XSS almacenado, path traversal y SSRF.
5. **Recomendar** validación robusta y almacenamiento seguro.

📖 Temas

#	Tema	Por qué importa
1	Validaciones de upload	Qué hay que evadir
2	Bypass por extensión	Filtros de blocklist
3	Bypass por Content-Type y magic bytes	Validación superficial
4	Web shell y RCE	Impacto máximo
5	Upload → XSS/SVG	Vectores alternativos
6	Path traversal en el nombre	Sobrescritura de archivos
7	Defensa: allowlist, renombrar, aislar	Cierre del fallo

📖 Definiciones y características

- **File upload inseguro:** aceptar archivos sin validar tipo, contenido o ubicación. Característica: puede llevar a RCE.

- **Web shell:** archivo (p. ej. `.php`) que ejecuta comandos al accederse. Característica: da control del servidor.
- **Magic bytes:** firma binaria inicial que identifica el tipo real. Característica: se puede falsificar para engañar validaciones.
- **Blocklist de extensiones:** prohibir ciertas extensiones. Característica: frágil; se evade con variantes (`.phtml`, `.php5`).
- **Allowlist:** permitir solo extensiones/tipos seguros. Característica: defensa robusta.
- **Almacenamiento fuera de webroot:** guardar uploads donde no se ejecuten. Característica: evita ejecución del contenido subido.

Herramientas y preparación

- **DVWA** (*File Upload*) y **PortSwigger labs** de file upload.
- **Burp** para modificar el nombre, extensión y Content-Type en la petición.
- Una web shell mínima de laboratorio (p. ej. PHP que ejecute `$_GET['cmd']`).

Laboratorio guiado

 Solo en labs propios y aislados.

1. En DVWA → *File Upload*, sube una imagen normal y observa dónde se guarda y cómo se sirve.
2. Sube una web shell PHP simple; si se bloquea por extensión, prueba variantes: `.phtml`, `.php5`, `.pHp`.
3. Evade validación por **Content-Type**: cambia el header a `image/png` en Burp manteniendo el contenido PHP.
4. Evade validación por **magic bytes**: antepone `GIF89a;` al código PHP.
5. Accede a la URL del archivo subido y ejecuta un comando (`?cmd=id`) en el lab.
6. Prueba vectores alternativos: sube un **SVG** con XSS, o un nombre con `../` para path traversal.
7. Documenta la validación evadida, el payload y el impacto.

Ejercicios

1. Enumera 5 extensiones alternativas que pueden ejecutar código PHP.
2. Evade una validación por Content-Type y otra por magic bytes.
3. Sube un SVG que ejecute JavaScript (XSS almacenado).
4. Usa `../` en el nombre para intentar sobrescribir un archivo.
5. Explica por qué guardar fuera del webroot mitiga el RCE.
6. Diseña una validación de upload robusta (allowlist + renombrado + escaneo).

Reto verificable

Consigue **RCE** subiendo una web shell en un lab (DVWA Medium o PortSwigger) evadiendo al menos una validación, y ejecuta un comando. **Criterio de aceptación:** entregas el archivo subido, la validación evadida, la evidencia de ejecución de comando y la corrección (allowlist, renombrado, almacenamiento fuera del webroot).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Extensión rechazada	Blocklist; prueba variantes o doble extensión
Se sube pero no ejecuta	Fuera del webroot o sin handler PHP; busca ubicación ejecutable
Content-Type validado	Falsifícalo en Burp manteniendo el contenido
Magic bytes comprobados	Antepón la firma del tipo permitido
Nombre saneado	Path traversal bloqueado; prueba encoding

Preguntas frecuentes

 **¿Basta validar la extensión?** No. Hay que validar tipo real, renombrar, y sobre todo almacenar donde no se ejecute el contenido.

 **¿Por qué un SVG es peligroso?** Porque es XML y puede contener JavaScript; servido inline, ejecuta XSS en el contexto de la app.

 **¿Dónde guardo los uploads?** Fuera del directorio web servible, con nombres generados, y sírvelos con Content-Type y `Content-Disposition` seguros.

Referencias

- Stuttard & Pinto, *The Web Application Hacker's Handbook*.
- OWASP File Upload Cheat Sheet.
- PortSwigger File upload vulnerabilities: <https://portswigger.net/web-security/file-upload>
- Li, *Bug Bounty Bootcamp*.

Siguiente clase

Clase 109 - Vulnerabilidades de logica de negocio